

Social Context — Part 3

Project: [OpenRefine](#)

Team members: Kingson Zhang, Mouhamed Osman, Zhenyu Song, Zian Xu

Snapshot date: 2026-05-11

We worked through these questions as a team. Every answer below had input from at least two of us, and we cross-checked the numbers together. Most facts come from the [OpenRefine repo](#), [GOVERNANCE.md](#), [CONTRIBUTING.md](#), the [openrefine.org](#) site, and the [community forum](#).

1. Stakeholders

1.1 Who are the people and/or organizations that are using this software?

Mostly data journalists, librarians, GLAM staff, researchers, and Wikidata editors. The repo topic tags (`datajournalism`, `opendata`, `reconciliation`, `wikidata`) match the forum traffic. Institutional users mentioned on [openrefine.org](#) include the Library of Congress, Wikimedia Deutschland, and the NYPL.

1.2 Why are they using this software (and not something else)?

OpenRefine is a local browser-based GUI with clustering for de-duplication and a reconciliation API for matching messy values against Wikidata, VIAF, etc. Competitors are cloud-only (Trifacta), spreadsheet-bound (Power Query), or code-only (`pandas`). The local-only part matters when users handle sensitive data.

1.3 Why is it open source and not proprietary?

It started as Freebase Gridworks, became Google Refine after Google bought Metaweb in 2010, and was open-sourced under BSD-3-Clause when Google dropped it in 2012.

1.4 Who owns the GitHub repository?

The OpenRefine GitHub organization. Since January 2020 the project has been fiscally sponsored by [Code for Science and Society](#), as documented in `GOVERNANCE.md`.

1.5 Who are the people and/or organizations that have invested their time in creating this software, and why?

Three groups:

- **Original engineers at Metaweb / Google (2009–2012):** dfhuynh and stefanom built Gridworks → Google Refine on company time.
- **Volunteer maintainers (2012–present):** tfmorris, wetneb (1,848 commits, top human contributor), thadguidry, Abbe98. Most use OpenRefine professionally (Wikimedia, libraries).
- **Paid roles via CS&S (2020–present):** magdmartin (Project Manager) and SoryRawyer (Release Manager), funded by donations and grants (`FUNDING.yml`).

1.6 What are the most important features and/or qualities that distinguish this software from the competition?

- Local, browser-based UI.
- GREL / Jython / Clojure expressions with an exportable, replayable operation history.
- Clustering (the subsystem we mapped in Assignment 2).
- The [Reconciliation API](#) — OpenRefine defines the spec.
- Plugin architecture; 30+ language translations via Weblate.

1.7 What does the media say about this software? What is its reputation? Has this reputation changed over time?

Strong and stable reputation in the library, journalism, and data-curation communities. It's a standard teaching tool in [Library Carpentry](#) and Programming Historian. Wider coverage dropped after Google handed it off in 2012, then recovered with the 3.x releases. 11.8k stars and 2.1k forks today.

1.8 What other components does it use and who owns them?

From `pom.xml` and `package.json`:

Layer	Major deps	Owner
Web server	Jetty	Eclipse

Layer	Major deps	Owner
JSON	Jackson	FasterXML
Office files	Apache POI, ODFDOM	Apache, Document Foundation
Expressions	Clojure, Jython, GREL, Velocity	Cognitect, Jython, OpenRefine, Apache
Frontend	jQuery, Select2	their OSS projects
Build / test	Maven, Cypress, ESLint, Prettier	Apache, Cypress.io , OpenJS

All open source. `butterfly` and `vicino` are project-specific forks kept in the org.

1.9 Have the primary developers changed over time or are they still involved (heavily or just a little bit)?

Substantially. The original Google authors stopped in 2012. `tfmorris` and `thadguidry` carried 2013–2020. `wetneb` led the 3.x releases. In 2025–2026, `SoryRawyer` took over release management and `tfmorris` is active again on triage. The project has survived several hand-offs, which is what `GOVERNANCE.md` is designed to support.

2. State of the project

2.1 What is the level of usage?

No telemetry (local-first). Proxies: 11,824 stars and 2,136 forks on the repo, hundreds of [Zenodo](#) citations, a busy forum, and 30+ Weblate translations.

2.2 What is the last release date?

3.10.1 on **2026-03-04** ([releases](#)).

2.3 What is the release roadmap, if there is one?

No single roadmap document. Implicit roadmap across [GitHub Milestones](#), project boards, and the `#dev` [forum](#). `GOVERNANCE.md` assigns roadmap stewardship to the Project Manager.

2.4 What is the number of pull requests submitted over the past month?

24 PRs between 2026-04-11 and 2026-05-11 ([PR search](#)). Most are Dependabot.

2.5 What is the number of accepted pull requests over the past month?

22 of the 24 merged, but only 2 are human-authored ([#7747](#) , [#7739](#)); the rest are bot dependency bumps.

2.6 Where is the 'hot bed' of activity today? How has that changed over time?

Three areas right now:

- Clustering refactor — [#7729](#) "Make clustering cancellable" continues our Part 2 issue.
- Infrastructure cleanup — Jetty 12 migration and REST API cleanup ([#7712](#) , [#7725](#)).
- Wikidata / Wikibase extension health.

Compared to 2018–2022, when Wikidata reconciliation features driven by `wetneb` dominated, the focus has shifted toward infrastructure.

2.7 How many open issues are there today, and how has that changed over the past 3 months?

682 open ([search](#)). Only ~12 of those were opened in the last 3 months, so the pile is basically flat (~30 new, ~10 closed in the last month).

2.8 How many older issues are there that are just not getting addressed or fixed?

A lot: 621 are older than 1 year, 560 older than 2 years. Only 7 open `Good First Issue` [s](#) left. The breakdown is 198 bugs vs 439 feature requests, so the long tail is mostly wishlist features.

2.9 Who submits the majority of the issues (e.g., developers, end-users)? Are they actual issues with the code, or usage issues?

Roughly two-thirds end-users (e.g. #7770 , #7760) and one-third maintainers (tfmorris 's #7712 , magdmartin 's #7761). They're real bugs, not usage questions — usage questions get routed to the forum by SUPPORT.md .

2.10 Is there a development forum and, if so, how responsive are the developers in answering questions?

Yes — forum.openrefine.org on Discourse, with #support , #dev , and #community categories. From the threads we read in April–May 2026, maintainers usually reply within a day (e.g. tfmorris reviewed Manishnemade12 's PR within 24h on April 30).

2.11 Does it appear that the documentation is up-to-date? How do you know?

Mostly. User docs live in a separate repo and are versioned per release; 3.10 docs shipped with 3.10. The [developer wiki](#) and an old 2014 extensions PDF are visibly stale — which matches what we ran into in Assignment 2.

2.12 What is the number of active developers?

~300 commits in the last year, 22 in the last 4 weeks ([contributors](#)). After stripping bots, roughly **5–10 active humans** in the last quarter.

3. Participation expectations

3.1 Who are the core maintainers?

From GOVERNANCE.md :

- **Core Developer Group:** tfmorris , Abbe98 .
- **Release Manager:** SorryRawyer .
- **Project Manager (paid):** magdmartin .
- **Advisory Committee:** Jan Ainali, Julie Faure-Lacroix, Esther Jackson.

3.2 Does the project accept contributions from outsiders?

Yes. `CONTRIBUTING.md` opens with "The OpenRefine project welcomes contributions in a variety of forms" and points new contributors to the [Good First Issue label](#). Progression goes User → Contributor → Committer → Core Dev.

3.3 Does the project follow any coding conventions?

- **Java:** auto-formatted by the Maven Formatter plugin, imports sorted by `import:sort`. Both run in `./refine lint`; CI fails without them.
- **JS / Cypress:** `eslint` + `prettier`.
- **Branch naming:** `<issue-number>-short-description` (per `CONTRIBUTING.md` and `copilot-instructions.md`).
- One issue per PR.

3.4 What is the typical process of contributing?

1. Pick an issue. For non-trivial work, post the design on `#dev` first.
2. Fork and branch from `master` using `<issue-number>-...`.
3. Implement, add tests (unit + Cypress for UI), run `./refine lint` and `./refine test`.
4. Open the PR. CI runs Java tests and Cypress; new contributors need a maintainer to approve the workflow.
5. A Core Dev reviews and merges; the Release Manager batches releases.

3.5 What has been the turnaround time for some recent pull requests?

Depends on the author. Dependabot merges in 1–8 days; recent human PRs took 14–20 days (`#7747` 14d, `#7739` 20d). Bigger architectural PRs sit longer — `#7729` has been open since 2026-03-24.

3.6 What have been some comments when recent pull requests have initially been rejected, but then been accepted?

From April–May 2026 review comments:

- **"Add a regression test."** PRs without one get paused (e.g. `#7771`).
- **"Out-of-scope changes."** Drop drive-by refactors.
- **"Run `./refine lint`."** Missing formatting is the most common cause of a red CI.

- **"Acknowledge the Copilot reviewer."** Copilot suggestions are treated as real review feedback.

3.7 What are the expectations of submitting test cases along with your code updates?

Tests are required. Both `CONTRIBUTING.md` and `copilot-instructions.md` say "Add unit tests and/or end-to-end tests for every bug fix or new feature." Java tests under `main/src/test/java/` ; Cypress tests under `main/tests/cypress/` .

3.8 What kinds of tools does the community expect you to use?

JDK 11+, Maven 3.8+, Node.js 18+, Cypress 15.14.x, ESLint + Prettier, Maven Formatter + import (all wired into `./refine lint`), and [Weblate](#) for UI translations.

4. AI use

4.1 Do the project's contribution guidelines address the use of AI? If not, have they stated anywhere why not? If so, summarize the important points of the guidelines.

No explicit AI policy in `CONTRIBUTING.md` or `GOVERNANCE.md` . Instead, on 2026-02-21 the project merged [#7667](#) , which adds `copilot-instructions.md` . That file tells the Copilot agent which build / test commands to use (`./refine` , `./refine test` , `./refine lint`), how to name branches, and that tests are mandatory. So the working policy: AI is fine, but the PR still has to pass the same review, lint, and test bar. A broader `AGENTS.md` proposal has been open since February 2026.

4.2 What is the percentage of AI-generated/AI-assisted contributions as of late? How can you tell (or not tell)?

Measurable: 11 PRs by the [Copilot SWE-agent account](#), 6 merged. Not measurable: how much of a typical human PR was drafted with Copilot / Cursor / ChatGPT — there's no required disclosure. So lower bound ~5% of recent merged PRs; upper bound unknown.

4.3 Does the project have its own AI agents (bots)? If so, what is the role of these agents (bots)?

Four bots in the repo:

Bot	Role
Dependabot	Weekly dependency PRs (config)
Copilot SWE-agent	Drafts PRs and reviews PRs; configured by <code>copilot-instructions.md</code>
Weblate	Pushes translation updates
Actions automations	<code>label_transfer.yml</code> copies labels; <code>unassign_issues.yml</code> un-assigns stale Good-First-Issue claims after 45 days

Only Copilot is real AI; the rest are rule-based.

5. Insights

5.1 List and briefly discuss the three things that surprised you the most about what you learned through this exercise?

- Bots do most of the merging.** 22 of 24 merged PRs last month were bots; Dependabot is the #2 all-time committer behind `wetneb` . The high merge count hides a small human team — which is why our Part 2 issue `#289` has been open since 2010.
- OpenRefine has effectively onboarded an AI teammate.** They merged `copilot-instructions.md` , 6 Copilot-agent PRs are in, and Copilot review comments are treated as real feedback. We expected caution; got pragmatism.
- The backlog is mostly feature requests, not bugs.** 439 features vs 198 bugs; 560 older than 2 years. The maintainers want help clearing this list — which is why our Part 2 issue `#7330` is labeled "Good First Issue".

5.2 List and briefly discuss the three things that surprised you the least about what you learned through this exercise?

1. **Heavy governance for a small team.** Apache-style meritocracy, 7-day VOTE windows, fiscal-sponsor rules — expected for a project that survived Metaweb → Google → community hand-offs.
2. **Tests and lint are non-negotiable.** Tests required per fix; reviewers send PRs back without them.
3. **Wikidata / reconciliation is the center of gravity.** Repo topics, reconciliation API, Wikibase extension, Wikimedia-funded contributors — all point to the same community.

5.3 Higher-level insight

OpenRefine is a community-rescued project running on a thin margin. Its survival since 2012 has come down to three things: BSD licensing (no vendor capture), explicit governance (survivable hand-offs), and recently automation + AI bots (so two Core Devs can manage a 700-issue backlog). The framing for our PRs: maintainers are responsive but stretched, so a clean, scoped, well-tested PR beats a clever one.